

Sudo and Privilege Management

Scenario

As a system administrator, you are tasked with managing user privileges for a team of engineers. Each user has different access requirements for administrative Exercises.

1. John: Needs full administrative privileges to perform any Exercise.
2. Mary: Can manage system updates but cannot modify user accounts or access sensitive files.
3. Paul: Can restart and manage specific services like Apache and MySQL but cannot make any other system changes.

Your Exercise is to configure sudo privileges to meet these requirements.

Create User Accounts

Commands Used:

```
sudo useradd -m john  
sudo useradd -m mary  
sudo useradd -m paul
```

Set Passwords:

```
sudo passwd john  
sudo passwd mary  
sudo passwd paul
```

Verified Users:

```
grep -E "john|mary|paul" /etc/passwd
```

```
kali-linux-2025.4-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
library
Type here to sea...
My Computer
kali-linux-2025.4-
Metasploitable2-
OWASP Broken W

Session Actions Edit View Help
(kali@olivil)-[~]
$ sudo useradd -m john
[sudo] password for kali:
(kali@olivil)-[~]
$ sudo useradd -m mary
(kali@olivil)-[~]
$ sudo useradd -m paul
(kali@olivil)-[~]
$ sudo passwd john
New password:
Retype new password:
passwd: password updated successfully
(kali@olivil)-[~]
$ sudo passwd mary
New password:
Retype new password:
passwd: password updated successfully
(kali@olivil)-[~]
$ sudo passwd paul
New password:
Retype new password:
passwd: password updated successfully
(kali@olivil)-[~]
$ grep -E "john|mary|paul" /etc/passwd
john:x:1009:1009::/home/john:/bin/sh
mary:x:1010:1010::/home/mary:/bin/sh
paul:x:1011:1011::/home/paul:/bin/sh
(kali@olivil)-[~]
$
```

The user accounts John, Mary, and Paul were created successfully, and secure passwords were assigned to each account. The `/etc/passwd` output confirmed that all three users were created with their own home directories and default login shells.

Exercise 2: Configure Sudo Privilege

Granted John Full Administrative Privileges

Commands Used For John:

1. Granted John Full Administrative Privileges by adding him to the sudo group
- `sudo usermod -aG sudo john`

Verifies John was successfully added to the sudo group

- `groups john`

Tested for John's sudo access by:

Switching to John's account

`su - john`

Then ran:

`sudo whoami`

I then exit john's account.

```
kali@olivil: ~  
  
(kali@olivil)-[~]  
$ sudo usermod -aG sudo john  
[sudo] password for kali:  
  
(kali@olivil)-[~]  
$ sudo usermod -aG sudo john  
  
(kali@olivil)-[~]  
$ groups john  
john : john sudo  
  
(kali@olivil)-[~]  
$ su - john  
Password:  
su: Authentication failure  
  
(kali@olivil)-[~]  
$ su - john  
Password:  
$ sudo whoami  
[sudo] password for john:  
root  
$ exit  
  
(kali@olivil)-[~]  
$
```

John was successfully added to the sudo group and granted full administrative privileges. After logging in as John, the sudo whoami command returned root, confirming that the configuration was successful.

2. Granted Mary Privileges to Manage System Updates

Commands Used:

- Edits the sudoers file:

sudo visudo

- Added the following rule at the end:

mary ALL=(ALL) NOPASSWD: /usr/bin/apt update, /usr/bin/apt upgrade
- Tested Mary's access:

su - mary sudo apt update # Should work without a password. sudo useradd testuser # Should fail.

3. Granted Paul Access to Manage Specific Services:

- Edits the sudoers file:

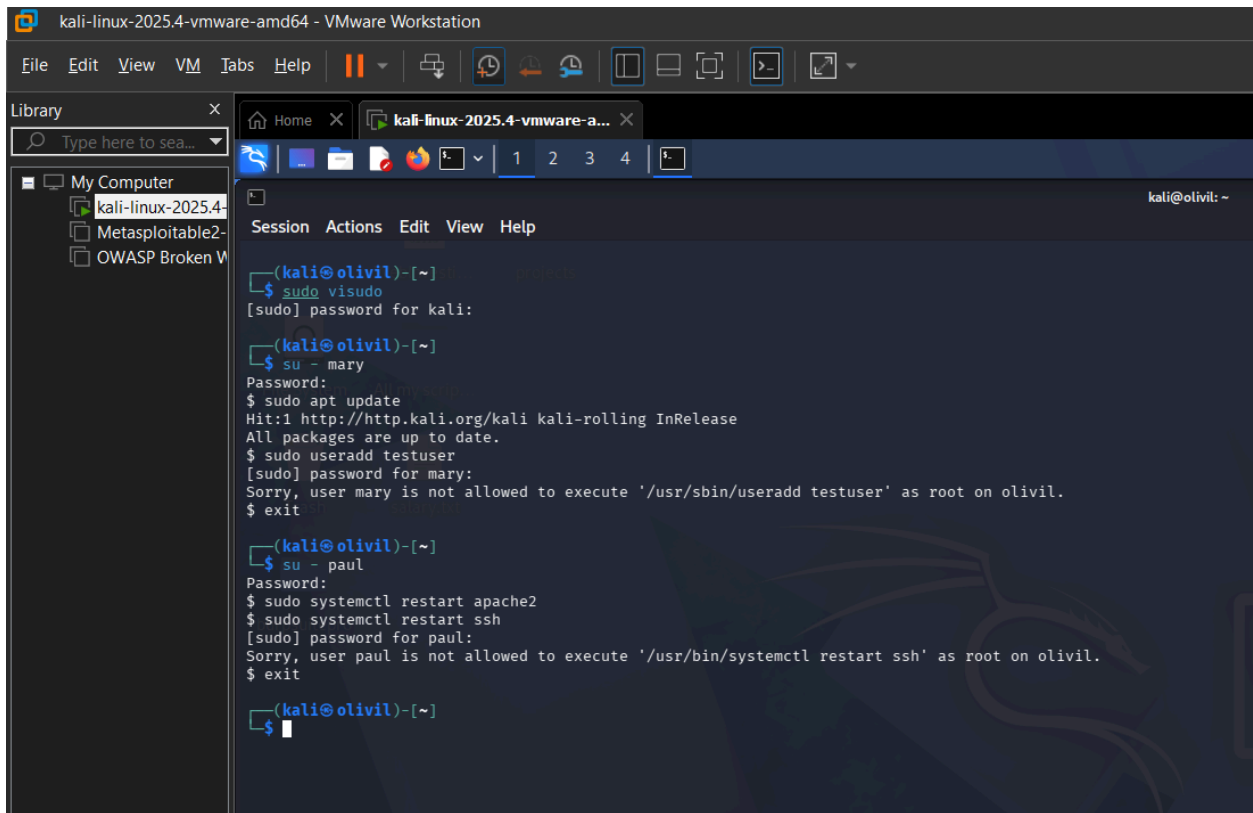
sudo visudo

- Add the following rule:

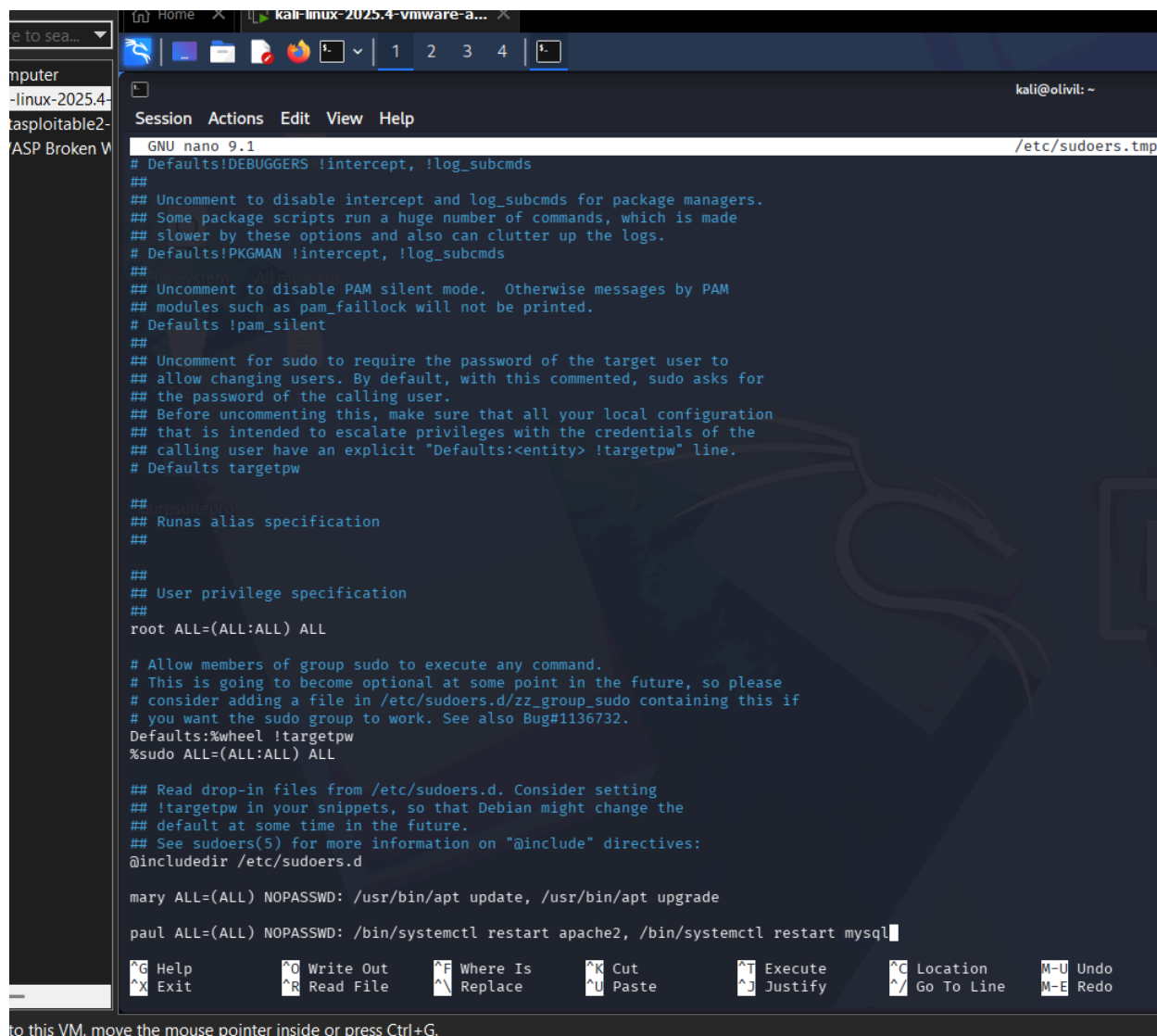
paul ALL=(ALL) NOPASSWD: /bin/systemctl restart apache2, /bin/systemctl restart mysql

- Test Paul's access:

su - paul sudo systemctl restart apache2
.sudo systemctl restart sshl.



```
kali-linux-2025.4-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
Library
Type here to search
My Computer
kali-linux-2025.4-
Metasploitable2-
OWASP Broken V
kali@olivil: ~
Session Actions Edit View Help
(kali@olivil)-[~]
$ sudo visudo
[sudo] password for kali:
(kali@olivil)-[~]
$ su - mary
Password:
$ sudo apt update
Hit:1 http://http.kali.org/kali kali-rolling InRelease
All packages are up to date.
$ sudo useradd testuser
[sudo] password for mary:
Sorry, user mary is not allowed to execute '/usr/sbin/useradd testuser' as root on olivil.
$ exit
(kali@olivil)-[~]
$ su - paul
Password:
$ sudo systemctl restart apache2
$ sudo systemctl restart ssh
[sudo] password for paul:
Sorry, user paul is not allowed to execute '/usr/bin/systemctl restart ssh' as root on olivil.
$ exit
(kali@olivil)-[~]
$
```



```
GNU nano 9.1 /etc/sudoers.tmp
# Defaults!DEBBUGERS !intercept, !log_subcmds
##
## Uncomment to disable intercept and log_subcmds for package managers.
## Some package scripts run a huge number of commands, which is made
## slower by these options and also can clutter up the logs.
# Defaults!PKGMAN !intercept, !log_subcmds
##
## Uncomment to disable PAM silent mode. Otherwise messages by PAM
## modules such as pam_faillock will not be printed.
# Defaults !pam_silent
##
## Uncomment for sudo to require the password of the target user to
## allow changing users. By default, with this commented, sudo asks for
## the password of the calling user.
## Before uncommenting this, make sure that all your local configuration
## that is intended to escalate privileges with the credentials of the
## calling user have an explicit "Defaults:<entity> !targetpw" line.
# Defaults targetpw

## Runas alias specification
##

## User privilege specification
##
root ALL=(ALL:ALL) ALL

# Allow members of group sudo to execute any command.
# This is going to become optional at some point in the future, so please
# consider adding a file in /etc/sudoers.d/zz_group_sudo containing this if
# you want the sudo group to work. See also Bug#1136732.
Defaults:%wheel !targetpw
%sudo ALL=(ALL:ALL) ALL

## Read drop-in files from /etc/sudoers.d. Consider setting
## !targetpw in your snippets, so that Debian might change the
## default at some time in the future.
## See sudoers(5) for more information on "@include" directives:
@includedir /etc/sudoers.d

mary ALL=(ALL) NOPASSWD: /usr/bin/apt update, /usr/bin/apt upgrade

paul ALL=(ALL) NOPASSWD: /bin/systemctl restart apache2, /bin/systemctl restart mysql
```

- Mary successfully ran apt update without being granted permission to manage user accounts. Her attempt to create testuser was denied, confirming that her sudo privileges were restricted as required.
- Paul successfully restarted the Apache service without a password, while his attempt to restart SSH was denied. This confirmed that his sudo privileges were limited to the specific services assigned to him.

Exercise 3: Restrict Access to the Sudo Command

1. Prevented unauthorized users from using sudo

- Ensured only specific users are part of the sudo

Commands Used:

getent group sudo

2. Removed a user from the Sudo

sudo deluser Ahmed sudo

```
kali@olivil: ~  
  
(kali@olivil)-[~]  
$ getent group sudo  
sudo:x:27:kali,olivil,john,Ahmed  
  
(kali@olivil)-[~]  
$ sudo deluser Ahmed sudo  
[sudo] password for kali:  
  
(kali@olivil)-[~]  
$ getent group sudo  
sudo:x:27:kali,olivil,john  
  
(kali@olivil)-[~]  
$
```

Ahmed was successfully removed from the sudo group. The final output confirmed that kali, olivil, and John remain members with sudo access.

Exercise 4: Logging and Monitoring Sudo Usage

1. Enabled sudo logs in using

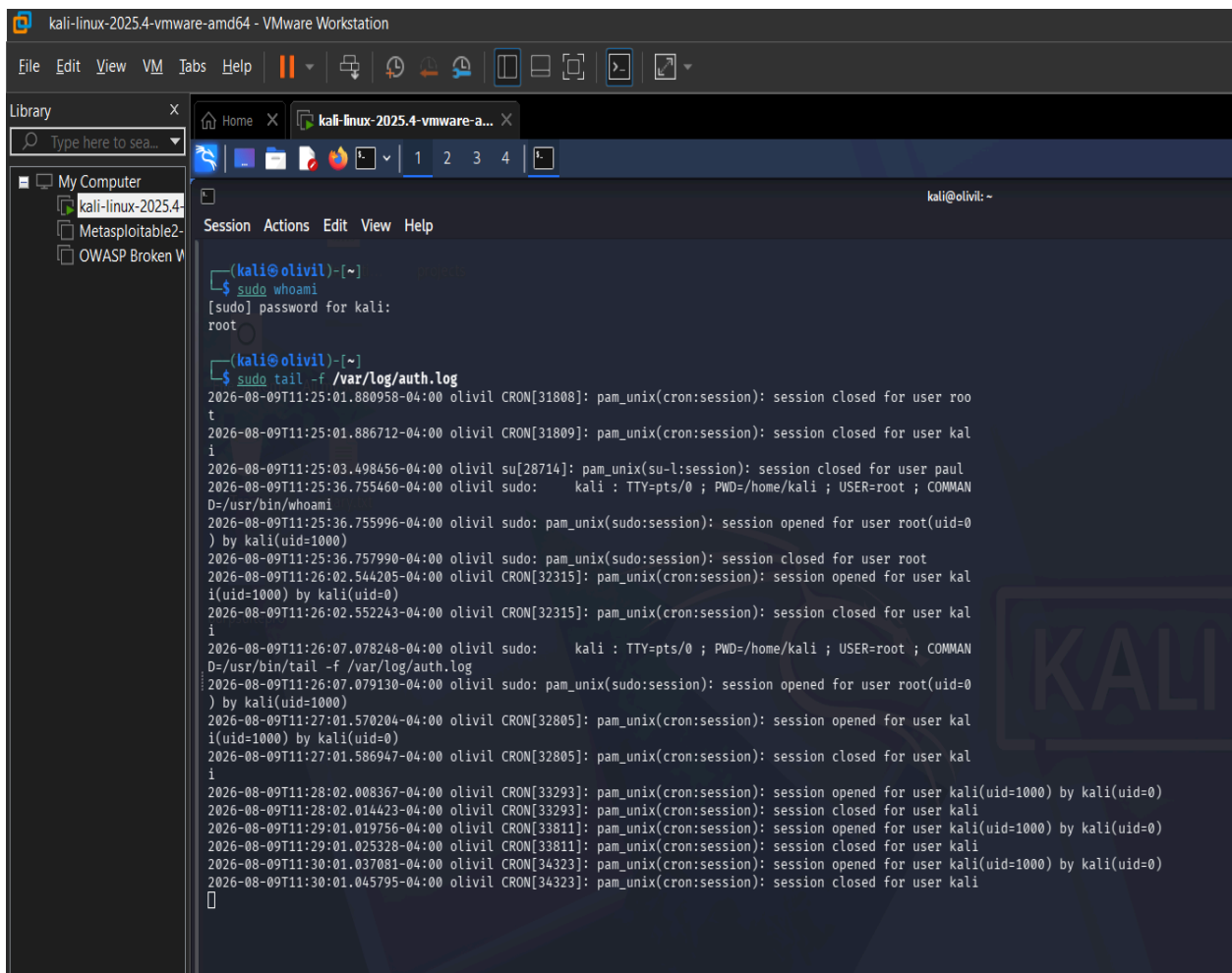
Commands Used:

/var/log/auth.log

2. Tested logging by running a command as a sudo user and checked the log.

sudo tail -f /var/log/auth.log

3. Look for entries indicating which commands were executed using sudo



```
kali@olivil:~$ sudo whoami
[sudo] password for kali:
root

kali@olivil:~$ sudo tail -f /var/log/auth.log
2026-08-09T11:25:01.880958-04:00 olivil CRON[31808]: pam_unix(cron:session): session closed for user root
2026-08-09T11:25:01.886712-04:00 olivil CRON[31809]: pam_unix(cron:session): session closed for user kali
2026-08-09T11:25:03.498456-04:00 olivil su[28714]: pam_unix(su-l:session): session closed for user paul
2026-08-09T11:25:36.755460-04:00 olivil sudo:    kali : TTY=pts/0 ; PWD=/home/kali ; USER=root ; COMMAND=/usr/bin/whoami
2026-08-09T11:25:36.755996-04:00 olivil sudo: pam_unix(sudo:session): session opened for user root(uid=0) by kali(uid=1000)
2026-08-09T11:25:36.757990-04:00 olivil sudo: pam_unix(sudo:session): session closed for user root
2026-08-09T11:26:02.544205-04:00 olivil CRON[32315]: pam_unix(cron:session): session opened for user kali(uid=1000) by kali(uid=0)
2026-08-09T11:26:02.552243-04:00 olivil CRON[32315]: pam_unix(cron:session): session closed for user kali
2026-08-09T11:26:07.078248-04:00 olivil sudo:    kali : TTY=pts/0 ; PWD=/home/kali ; USER=root ; COMMAND=/usr/bin/tail -f /var/log/auth.log
2026-08-09T11:26:07.079130-04:00 olivil sudo: pam_unix(sudo:session): session opened for user root(uid=0) by kali(uid=1000)
2026-08-09T11:27:01.570204-04:00 olivil CRON[32805]: pam_unix(cron:session): session opened for user kali(uid=1000) by kali(uid=0)
2026-08-09T11:27:01.586947-04:00 olivil CRON[32805]: pam_unix(cron:session): session closed for user kali
2026-08-09T11:28:02.008367-04:00 olivil CRON[33293]: pam_unix(cron:session): session opened for user kali(uid=1000) by kali(uid=0)
2026-08-09T11:28:02.014423-04:00 olivil CRON[33293]: pam_unix(cron:session): session closed for user kali
2026-08-09T11:29:01.019756-04:00 olivil CRON[33811]: pam_unix(cron:session): session opened for user kali(uid=1000) by kali(uid=0)
2026-08-09T11:29:01.025328-04:00 olivil CRON[33811]: pam_unix(cron:session): session closed for user kali
2026-08-09T11:30:01.037081-04:00 olivil CRON[34323]: pam_unix(cron:session): session opened for user kali(uid=1000) by kali(uid=0)
2026-08-09T11:30:01.045795-04:00 olivil CRON[34323]: pam_unix(cron:session): session closed for user kali
```

The sudo command was successfully executed as olivil, returning root. The /var/log/auth.log output also recorded the sudo whoami command and the corresponding root session, confirming that sudo activity is being logged successfully

How the Sudoers File Is Used to Manage Privileges

The sudoers file is used to control which users can run administrative commands and what level of access they have. In this lab, it was used to give John full administrative privileges, while Mary and Paul were given limited access to only the specific commands required for their tasks. This helps improve system security by following the principle of least privilege and preventing unauthorized system changes.

Securing Network Services and Ports

1. List Open Ports.

Command Used:

```
sudo ss -tln
```

2. Closed Unnecessary Ports by:

- Stopping an unnecessary service Apache:

```
sudo systemctl stop apache2 sudo systemctl disable apache2
```

```
sudo ls -l /etc/sudoers /etc/sudoers.backup
```

5. Check Apache installation

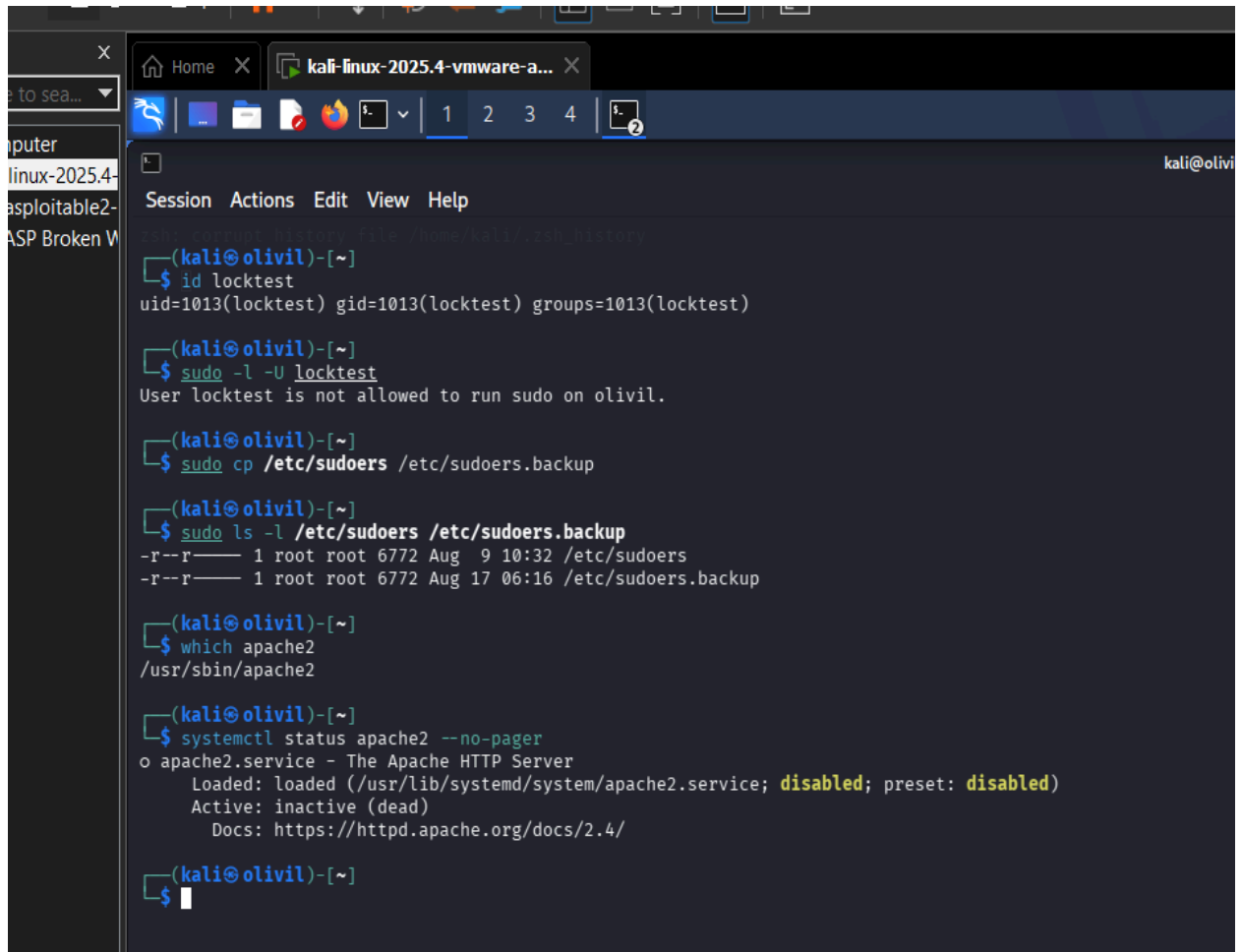
which apache2

6. Check Apache status

```
systemctl status apache2 --no-pager
```

7. Safely edit sudoers

```
sudo visudo
```



```
kali@kali:~$ id locktest
uid=1013(locktest) gid=1013(locktest) groups=1013(locktest)

kali@kali:~$ sudo -l -U locktest
User locktest is not allowed to run sudo on olivil.

kali@kali:~$ sudo cp /etc/sudoers /etc/sudoers.backup

kali@kali:~$ sudo ls -l /etc/sudoers /etc/sudoers.backup
-r--r--r-- 1 root root 6772 Aug  9 10:32 /etc/sudoers
-r--r--r-- 1 root root 6772 Aug 17 06:16 /etc/sudoers.backup

kali@kali:~$ which apache2
/usr/sbin/apache2

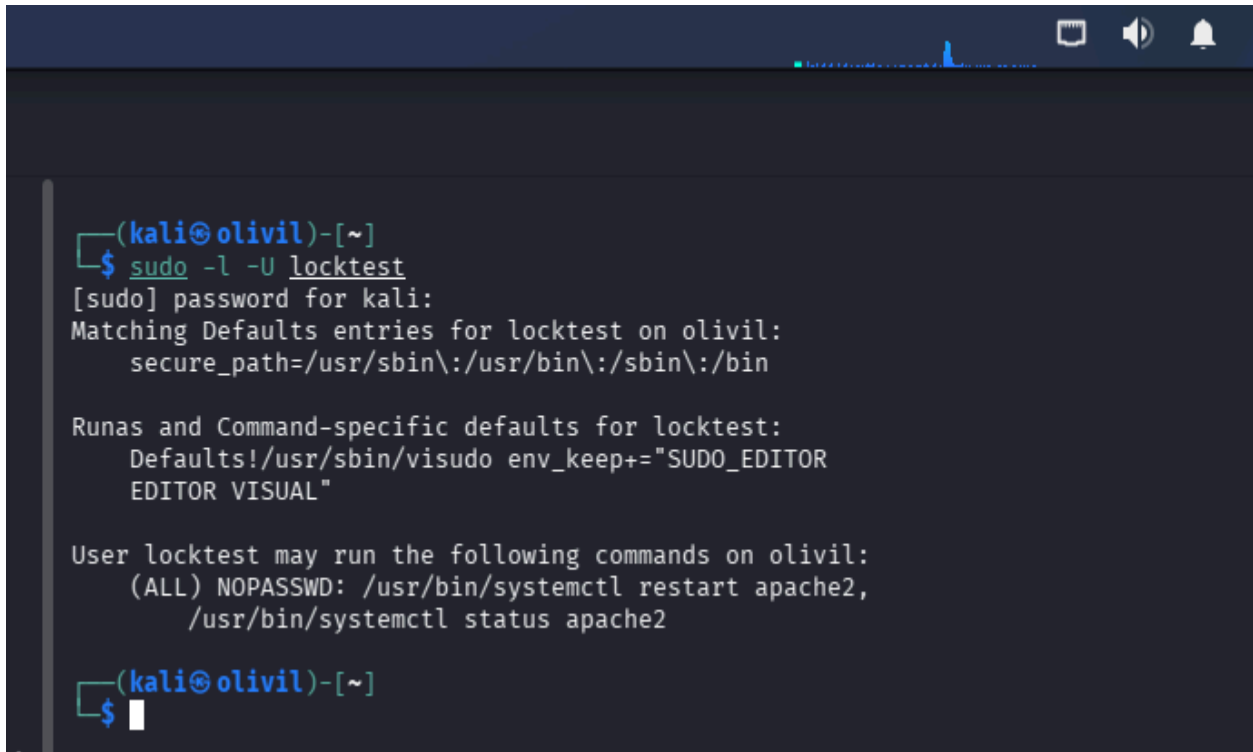
kali@kali:~$ systemctl status apache2 --no-pager
o apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; disabled; preset: disabled)
   Active: inactive (dead)
   Docs: https://httpd.apache.org/docs/2.4/

kali@kali:~$
```

```
kali@olivil: ~  
  
(kali@olivil)-[~]  
$ sudo visudo  
[sudo] password for kali:  
  
(kali@olivil)-[~]  
$  
  
kali@olivil: ~  
  
GNU nano 9.1 /etc/sudoers.tmp *  
## Some package scripts run a huge number of commands, which is made  
## slower by these options and also can clutter up the logs.  
# Defaults!PKGMAN !intercept, !log_subcmds  
##  
## Uncomment to disable PAM silent mode. Otherwise messages by PAM  
## modules such as pam_faillock will not be printed.  
# Defaults !pam_silent  
##  
## Uncomment for sudo to require the password of the target user to  
## allow changing users. By default, with this commented, sudo asks for  
## the password of the calling user.  
## Before uncommenting this, make sure that all your local configuration  
## that is intended to escalate privileges with the credentials of the  
## calling user have an explicit "Defaults:<entity> !targetpw" line.  
# Defaults targetpw  
##  
## Runas alias specification  
##  
##  
## User privilege specification  
##  
root ALL=(ALL:ALL) ALL  
  
# Allow members of group sudo to execute any command.  
# This is going to become optional at some point in the future, so please  
# consider adding a file in /etc/sudoers.d/zz_group_sudo containing this if  
# you want the sudo group to work. See also Bug#1136732.  
Defaults:%wheel !targetpw  
#sudo ALL=(ALL:ALL) ALL  
  
## Read drop-in files from /etc/sudoers.d. Consider setting  
## !targetpw in your snippets, so that Debian might change the  
## default at some time in the future.  
## See sudoers(5) for more information on "@include" directives:  
@includedir /etc/sudoers.d  
  
nary ALL=(ALL) NOPASSWD: /usr/bin/apt update, /usr/bin/apt upgrade  
  
aul ALL=(ALL) NOPASSWD: /bin/systemctl restart apache2, /bin/systemctl restart mysql  
  
locktest ALL=(ALL) NOPASSWD: /usr/bin/systemctl restart apache2, /usr/bin/systemctl status apache2  
  
G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo      M-A Set M  
X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line  M-E Redo     M-6 Copy
```

8. Verify the sudo rule

sudo -l -U locktest



```
(kali@olivil)-[~]
$ sudo -l -U locktest
[sudo] password for kali:
Matching Defaults entries for locktest on olivil:
    secure_path=/usr/sbin\:/usr/bin\:/sbin\:/bin

Runas and Command-specific defaults for locktest:
Defaults!/usr/sbin/visudo env_keep+="SUDO_EDITOR
EDITOR VISUAL"

User locktest may run the following commands on olivil:
    (ALL) NOPASSWD: /usr/bin/systemctl restart apache2,
                  /usr/bin/systemctl status apache2

(kali@olivil)-[~]
$
```

it shows:

(ALL) NOPASSWD: /usr/bin/systemctl restart apache2, /usr/bin/systemctl status apache2

9. Switch to locktest

su - locktest

10. Confirm you are actually locktest

whoami

11. Check the user's sudo permissions

sudo -l

12. Test Apache status command

sudo systemctl status apache2 --no-pager

13. Test Apache restart command

`sudo systemctl restart apache2`

14. Test SSH command

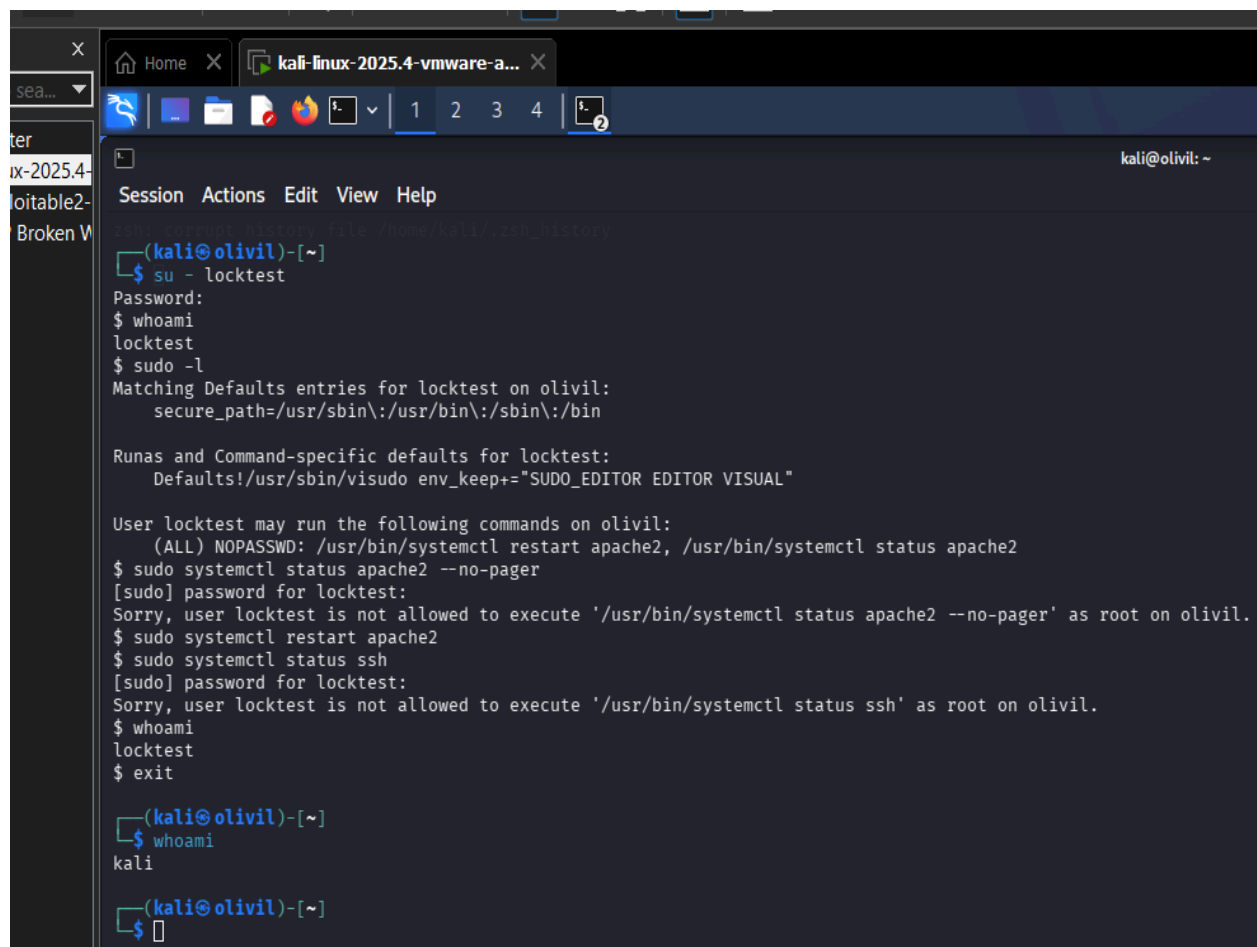
`sudo systemctl status ssh`

4. Returned to kali

`exit`

Then:

`whoami`



```
(kali@olivil)-[~]
$ su - locktest
Password:
$ whoami
locktest
$ sudo -l
Matching Defaults entries for locktest on olivil:
    secure_path=/usr/sbin\:/usr/bin\:/sbin\:/bin

Runas and Command-specific defaults for locktest:
    Defaults!/usr/sbin/visudo env_keep+="SUDO_EDITOR EDITOR VISUAL"

User locktest may run the following commands on olivil:
    (ALL) NOPASSWD: /usr/bin/systemctl restart apache2, /usr/bin/systemctl status apache2
$ sudo systemctl status apache2 --no-pager
[sudo] password for locktest:
Sorry, user locktest is not allowed to execute '/usr/bin/systemctl status apache2 --no-pager' as root on olivil.
$ sudo systemctl restart apache2
$ sudo systemctl status ssh
[sudo] password for locktest:
Sorry, user locktest is not allowed to execute '/usr/bin/systemctl status ssh' as root on olivil.
$ whoami
locktest
$ exit

(kali@olivil)-[~]
$ whoami
kali

(kali@olivil)-[~]
$
```

I configured fine-grained sudo permissions for the locktest account using visudo. The account was granted passwordless access to restart and check the

status of Apache, while unauthorized commands were restricted. I tested the permissions using the `locktest` account and verified the sudo configuration successfully. After completing the exercise, Apache was stopped and returned to its previous disabled state

Question 4.1: Why is it recommended to use `visudo` instead of directly editing `/etc/sudoers`?

`visudo` checks the sudoers file for syntax errors before saving, helping prevent mistakes that could disable sudo access or lock administrators out.

Question 4.2: What are the security implications of using `NOPASSWD` in sudo configurations?

`NOPASSWD` improves convenience but reduces security because commands can be run with elevated privileges without password verification. If an account is compromised, an attacker may immediately gain access to the allowed privileged commands.

Part 5: Security Compliance and Best Practices

Exercise 5.1: Perform a Security Audit Checklist

Created a script to automate security checks

Commands Used:

```
cd ~  
nano security_audit.sh
```